CONFIDENTIAL

SECURITY ASSESSMENT

Penetration Testing Report

HF2019 Linux

Network, Web Application, and Host Privilege Review

Internal laboratory assessment • Target 10.0.2.6

ASSESSMENT DATE 19 October 2026 **TARGET HOST** HF2019-Linux — 10.0.2.6 **ASSESSMENT SOURCE** Kali Linux — 10.0.2.12	**REPORT VERSION** 1.0 **REPORT STATUS** Final **CLASSIFICATION** Confidential — Authorized Lab

Prepared by

Aswin Sunil

Handling Notice
This report contains technical attack-path details and references to recovered laboratory credentials and challenge flags. Restrict distribution to authorized recipients only. Redact all secrets and flags before sharing this document outside the lab environment.

Contents

1 Document Control3

2 Executive Summary4

3 Scope5

4 Rules of Engagement5

5 Methodology7

6 Target Information7

7 Reconnaissance & Enumeration9

8 Attack Path & Narrative10

9 Detailed Findings11

10 Initial Access14

11 Privilege Escalation14

12 Post-Exploitation14

13 Security Impact15

14 Remediation Plan15

15 Retest Recommendations16

16 Conclusion16

17 Appendices17

APPENDICES

A Command & Evidence Register17

B Sensitive Evidence Handling17

C Severity Definitions Used17

D Limitations & Assumptions18

E Restricted Lab Flags18

1 Document Control

Document title	HF2019 Linux Penetration Testing Report
Document ID	HF2019-PT-2026-001
Version	1.0
Status	Final
Assessment date	19 October 2026
Target environment	Virtualized training / intentionally vulnerable lab (as represented by the supplied evidence).
Distribution	Authorized lab owner and assessment stakeholders.
Evidence basis	Terminal output supplied during the assessment.
Limitations	No independent access to the target, raw scan files, or system logs was performed by the report author. Findings are based on the evidence supplied.

Revision History

Version	Date	Author	Change summary
1.0	19 Oct 2026	Aswin Sunil	Initial final report compiled from the recorded lab walkthrough.

2 Executive Summary

The HF2019 Linux assessment demonstrated a complete compromise of the target host at 10.0.2.6. The observed attack chain began with anonymous FTP access, which exposed WordPress application files including wp-config.php. Web enumeration identified WordPress 5.2.3 and the wp-google-maps plugin. A Metasploit auxiliary SQL injection module retrieved the WordPress webmaster account hash; offline password recovery disclosed a password that was subsequently accepted by SSH. After logging in as webmaster, sudo policy inspection showed unrestricted command execution as any user, enabling immediate transition to root.

The most significant risk is the combination of unauthenticated application-layer exposure, recoverable and reused credentials, and unrestricted sudo permissions. In a comparable production environment, this chain could allow an external or adjacent attacker to move from unauthenticated network access to full host compromise, subject to network reachability and environmental controls.

Assessment at a Glance

Measure	Result
Target	10.0.2.6 (HF2019-Linux)
Open TCP ports observed	21, 22, 80, 10000
Confirmed critical outcome	Root-level command execution achieved
Detailed findings	3 primary findings (2 High, 1 Critical)
Primary risk themes	Sensitive file exposure; SQL injection / credential compromise; excessive privilege
Additional hardening observations	Outdated WordPress; directory listing; XML-RPC exposure; Webmin service

Overall Assessment
High risk in the tested lab configuration. The observed chain culminated in root access. Severity labels in this report are qualitative and are not presented as CVSS scores.

Finding Severity Overview

ID	Finding	Severity
F01	Anonymous FTP exposes WordPress configuration and database secret	HIGH
F02	WordPress Google Maps SQL injection → account-hash disclosure → reused credential enables SSH access	HIGH
F03	Unrestricted sudo permissions allow webmaster to become root	CRITICAL

3 Scope

3.1 In-Scope Asset

Hostname	HF2019-Linux
IPv4 address	10.0.2.6
Attacker host	Kali Linux — 10.0.2.12
Network context	10.0.2.0/24; VirtualBox-style virtual NIC observed
Protocols / services assessed	FTP, SSH, HTTP/WordPress, Webmin HTTP-over-TLS

3.2 Activities Performed

- Local network discovery using ARP.
- TCP port and service-version enumeration using Nmap.
- Anonymous FTP access and file retrieval.
- HTTP technology fingerprinting and manual endpoint checks.
- WordPress enumeration with WPScan.

- **Validation of the wp-google-maps SQL injection path using a Metasploit auxiliary module.**
- **Offline password-hash recovery with John the Ripper.**
- **SSH authentication testing with recovered lab credentials.**
- **Local privilege review using sudo -l and validation of root access.**

3.3 Out of Scope / Not Performed

- **Denial-of-service or stress testing.**
- **Persistence installation, destructive actions, or data modification.**
- **Broad lateral movement beyond the target host.**
- **Full Webmin vulnerability assessment or exploitation.**
- **Comprehensive review of all upload-directory contents.**
- **Independent forensic review of target logs or verification of remediation.**

4 Rules of Engagement

The following rules are documented from the apparent lab context and should be confirmed by the system owner before this report is used as a formal engagement record. No separate signed rules-of-engagement document was supplied.

- **Authorization: Testing is assumed to have been conducted against an intentionally vulnerable machine controlled by the tester or lab owner.**
- **Target limitation: Activity described in this report is limited to 10.0.2.6 and its exposed services.**
- **Permitted techniques: Discovery, service enumeration, application testing, credential-hash recovery, SSH access validation, and privilege validation as recorded.**
- **Safety: No denial-of-service, destructive testing, persistence, or broad lateral movement was reported.**
- **Evidence handling: Terminal output and recovered secrets are treated as confidential lab evidence.**
- **Stop condition: Testing should stop after objectives are achieved or if activity affects systems outside the authorized lab.**

5 Methodology

The assessment followed a practical, evidence-driven penetration-testing workflow aligned with common industry phases. This was a focused lab exercise, not a claim of full compliance with a particular standard.

Phase	Techniques / tools	Purpose
Discovery	ip, ip route, ip neigh, arp-scan	Establish attacker network configuration and identify target hosts.
Service enumeration	Nmap -sC -sV	Identify open TCP ports, service banners, and default-script findings.
Web enumeration	WhatWeb, curl, WPScan, manual source review	Identify WordPress, versions, plugin paths, and exposed directories/endpoints.
Validation	Metasploit auxiliary module	Validate the observed SQL injection path and retrieve the WordPress account hash.
Credential analysis	John the Ripper	Assess whether the recovered hash could be recovered with a wordlist.
Access validation	SSH	Confirm whether the recovered credentials permitted remote login.
Privilege validation	sudo -l, sudo su	Determine effective local privilege and verify root access.

Evidence Standard
A finding is treated as confirmed when the supplied terminal output directly demonstrates the condition or its impact. Fingerprints with confidence qualifiers and services not further tested are recorded as observations rather than proven exploitable vulnerabilities.

6 Target Information

Attribute	Observed value
Hostname	HF2019-Linux
IP address	10.0.2.6
Operating system	Debian Linux family (service and SSH banner identification)
Kernel	4.19.0-0.bpo.6-amd64
Architecture	x86_64
FTP server	vsftpd 3.0.3
SSH server	OpenSSH 7.4p1 Debian 10+deb9u7
Web server	Apache httpd 2.4.25 (Debian)
CMS	WordPress 5.2.3
WordPress theme	Twenty Seventeen 2.2 (WPScan confidence 80%)
WordPress plugin	wp-google-maps (exact version not identified)
Administrative web service	MiniServ 1.890 / Webmin on TCP 10000

7 Reconnaissance & Enumeration

7.1 Network Enumeration

The attacker interface was eth0 with IPv4 address 10.0.2.12/24. The route table showed the directly connected network 10.0.2.0/24 and default gateway 10.0.2.1.

```
ip -br addr
ip route
sudo arp-scan --localnet
```

ARP discovery returned three responding hosts: 10.0.2.1, 10.0.2.2, and 10.0.2.6. The target at 10.0.2.6 returned MAC address 08:00:27:d9:8e:69, consistent with a VirtualBox virtual NIC prefix.

7.2 Service Enumeration

```
nmap -sC -sV -oN hf2019-initial.txt 10.0.2.6
```

Port	Service	Observed version / result
21/tcp	FTP	vsftpd 3.0.3; anonymous login allowed; WordPress files listed
22/tcp	SSH	OpenSSH 7.4p1 Debian 10+deb9u7
80/tcp	HTTP	Apache 2.4.25 (Debian); WordPress 5.2.3
10000/tcp	HTTPS/HTTP	MiniServ 1.890; Webmin login page identified

Nmap reported the other 996 ports in its default scan range as closed. Webmin was discovered but not exploited during the recorded path.

7.3 Web Enumeration

Check	Observation	Assessment note
WhatWeb	Apache 2.4.25, Debian, WordPress 5.2.3	Technology fingerprint; version disclosure.
/robots.txt	404 Not Found	No robots.txt present at this path; not a vulnerability by itself.
/wp-login.php	200 OK; WordPress test cookie set	Login endpoint is reachable.
WPScan	WordPress 5.2.3; Twenty Seventeen theme; wp-google-maps plugin	Plugin version not determined; no API token, so vuln-database data was omitted.
/xmlrpc.php	Reported enabled by WPScan	Potentially unnecessary exposed endpoint; not exploited.
/wp-content/uploads/	Apache directory listing enabled	Directory structure exposed; only observed paths documented.
/wp-content/uploads/2019/	Directory listing showed 09/	Contents below 2019/09/ were not fully examined.
/readme.html	Public WordPress readme detected	Version / product disclosure.

8 Attack Path & Narrative

The following sequence summarizes how the assessment progressed from initial discovery to root-level access.

- **Host discovery identified 10.0.2.6 as the HF2019 Linux target.**
- **Nmap identified FTP, SSH, HTTP, and Webmin services.**
- **Anonymous FTP login succeeded and exposed WordPress files, including wp-config.php.**
- **The downloaded wp-config.php disclosed database connection details and confirmed the wp_ table prefix.**
- **Web enumeration identified WordPress 5.2.3 and the wp-google-maps plugin.**
- **The Metasploit wp_google_maps_sqli auxiliary module retrieved the webmaster account hash from the WordPress users table.**
- **John the Ripper recovered the plaintext password from the hash using a wordlist.**
- **The recovered password was accepted for SSH as webmaster, confirming credential reuse across services.**
- **sudo -l showed that webmaster could run (ALL) ALL; sudo su yielded a root shell.**
- **The user-level and root-level flags were read, confirming access at both privilege levels.**

Attack-Chain Outcome
Unauthenticated network access progressed to authenticated remote shell access and then root-level command execution.

9 Detailed Findings

Finding HF2019-F01 Anonymous FTP Exposes WordPress Configuration	HIGH

Finding ID	HF2019-F01
Severity	High
Status	Confirmed
Affected asset	10.0.2.6 — FTP service / WordPress application files
Evidence basis	Nmap reported anonymous FTP login allowed. FTP directory listing exposed wp-config.php; download completed successfully.

Description

The FTP service accepted an anonymous login and exposed WordPress application files. The tester downloaded wp-config.php and extracted the database name, username, password, host, and table prefix. The database password is intentionally redacted in this report body because it is a secret and should be rotated.

Impact

- Disclosure of application configuration and database credentials.
- Potential unauthorized database access if the exposed credentials are usable from an accessible database endpoint or local application context.
- Additional information that can support subsequent compromise of the web application.

Evidence

```
ftp 10.0.2.6
Name: anonymous
230 Login successful.
ls -la
-rw-rw-r-- 1 ftp ftp 3255 Sep 27 2019 wp-config.php
get wp-config.php
grep -nE "DB_NAME|DB_USER|DB_PASSWORD|DB_HOST|table_prefix" wp-config.php
DB_NAME: wordpress
DB_USER: wordpress
DB_HOST: localhost
table_prefix: wp_
```

Recommendation

- Disable anonymous FTP access.
- Prevent FTP users from reading application configuration files; use least-privilege filesystem ownership and permissions.
- Rotate the exposed database credential and review database logs for unauthorized access.
- Use a database account limited to the privileges required by WordPress.

Finding HF2019-F02 WordPress SQL Injection & Credential Compromise	HIGH

Finding ID	HF2019-F02
Severity	High
Status	Confirmed
Affected asset	10.0.2.6 — WordPress 5.2.3 / wp-google-maps plugin; SSH account webmaster
Evidence basis	Metasploit module retrieved the webmaster PHPass hash; John the Ripper recovered a password; SSH login succeeded with that password.

Description

WPScan and page-source review identified the wp-google-maps plugin. The Metasploit auxiliary module auxiliary/admin/http/wp_google_maps_sqli returned the webmaster account hash from the WordPress users table. John the Ripper recovered the password using a wordlist. The recovered password was then accepted by the SSH service, demonstrating credential reuse.

Impact

- Unauthenticated SQL injection enabled disclosure of a WordPress password hash.
- A weak password was recoverable through offline wordlist testing.
- Password reuse converted a web-application compromise into remote shell access as webmaster.
- The resulting account access enabled subsequent privilege escalation because of the sudo configuration described in Finding HF2019-F03.

Evidence

```
WPScan: WordPress 5.2.3; wp-google-maps plugin detected
Metasploit: auxiliary/admin/http/wp_google_maps_sqli
Result: Found webmaster <PHPass hash> webmaster@none.local
John --show: 1 password hash cracked
SSH: ssh webmaster@10.0.2.6 — login successful
```

Sensitive Evidence Handling
The recovered hash and plaintext password are omitted from the main report body. Store them only in a restricted evidence repository if retention is required; rotate the password immediately in any non-disposable environment.

Recommendation

- Update WordPress core and update, replace, or remove the vulnerable plugin version after verifying a fixed release.
- Review vendor advisories and confirm the exact plugin version on the target.
- Rotate the webmaster and any reused credentials; require unique passwords across WordPress and SSH.
- Enforce a strong password policy and MFA for web administration where supported.

- Restrict SSH exposure and prefer managed SSH keys with password authentication disabled where feasible.
- Review WordPress, web server, and SSH logs for signs of prior unauthorized access.

Finding HF2019-F03 **Unrestricted Sudo Allows Root Access**	**CRITICAL**

Finding ID	**HF2019-F03**
Severity	Critical
Status	Confirmed
Affected asset	10.0.2.6 — local sudo policy for webmaster
Evidence basis	sudo -l displayed (ALL) ALL; sudo su successfully changed the shell to root.

Description

After obtaining a shell as webmaster, sudo -l showed that the account could run all commands as all users. The tester executed sudo su and obtained a root shell. This is a direct privilege-control failure; no kernel exploit was required.

Impact

- Any attacker who compromises webmaster can obtain full administrative control of the host.
- Root access permits reading and modifying system files, accessing other local users' data, changing security controls, and potentially establishing persistence.
- The privilege boundary between a standard service/application account and the system administrator is effectively absent.

Evidence

```
sudo -l
User webmaster may run the following commands on localhost:
(ALL) ALL
sudo su
root@HF2019-Linux:/#
```

Recommendation

- Remove unrestricted sudo permissions from webmaster unless there is a documented and approved operational need.
- Grant only the exact commands required, with fixed arguments and no shell-escape capability.
- Audit /etc/sudoers and /etc/sudoers.d/ using visudo.
- Separate web-application administration from operating-system administration.
- Monitor sudo usage and review authentication and privilege-escalation logs.

10 Initial Access

Initial interactive access was obtained through SSH after recovering the webmaster account password from a WordPress hash. The successful login confirmed that the same password was

reused between the WordPress account and the SSH account.

```
ssh webmaster@10.0.2.6
# Password entered interactively; do not store plaintext credentials in command history
```

The SSH banner identified the host as HF2019-Linux and reported Debian kernel 4.19.0-0.bpo.6-amd64. The client also displayed a post-quantum key-exchange warning; this warning did not block login and was not the root cause of compromise.

11 Privilege Escalation

Privilege escalation was achieved through misconfigured sudo permissions rather than exploitation of a kernel vulnerability. The sudo policy explicitly permitted all commands as all users.

```
sudo -l
sudo su
id
```

The supplied evidence shows the shell prompt changed to root. The id command is included as a recommended verification step for a repeatable retest; its output was not included in the supplied evidence.

12 Post-Exploitation

Post-exploitation activity was limited to verifying access and reading the two lab flags. No persistence, destructive modification, data exfiltration beyond the flags, or lateral movement was reported.

Check	Observed result
Webmaster home directory	Contained flag.txt readable by webmaster
User-level flag	Captured; value intentionally omitted from the main body
Root directory	/root contained flag.txt
Root-level flag	Captured; value intentionally omitted from the main body
Persistence / cleanup	No persistence activity reported; VM restoration not independently verified

13 Security Impact

The demonstrated chain shows that weaknesses across different layers compounded into full host compromise. FTP exposure provided sensitive application configuration; the WordPress SQL injection exposed a password hash; weak password selection and reuse enabled SSH access; and unrestricted sudo converted that foothold into root access.

Impact area	Potential consequence
Confidentiality	Disclosure of application configuration, database secrets, WordPress account data, and local files accessible to the compromised accounts.
Integrity	A root-level attacker could modify application content, system configuration, accounts, and security controls.
Availability	A root-level attacker could disrupt services or damage the host; no availability-impacting action was performed during this test.
Credential security	The observed chain demonstrates password-hash exposure, offline recovery, and credential reuse.
Privilege separation	Unrestricted sudo eliminated the boundary between the compromised account and root.

14 Remediation Plan

Priority	Action	Suggested owner	Target
P0 — Immediate	Remove unrestricted sudo access from webmaster; verify least privilege.	Linux / system administrator	Immediately
P0 — Immediate	Rotate the exposed database credential and all affected / reused account passwords.	Application & system owners	Immediately
P1 — High	Disable anonymous FTP and restrict access to application files, especially wp-config.php.	Infrastructure administrator	Within 7 days
P1 — High	Patch WordPress and the vulnerable Google Maps plugin; remove unsupported components.	WordPress administrator	Within 7 days
P1 — High	Review web, FTP, SSH, database, and sudo logs for signs of prior compromise.	Security operations	Within 7 days
P2 — Medium	Disable directory indexing on uploads and remove unnecessary exposed files.	Web administrator	Within 14 days
P2 — Medium	Review XML-RPC necessity and restrict or disable if not required.	WordPress administrator	Within 14 days
P2 — Medium	Restrict Webmin to trusted management networks and update to a supported version.	Infrastructure administrator	Within 14 days
P2 — Medium	Harden SSH: reduce exposure, use managed keys, disable password login where feasible.	Infrastructure administrator	Within 30 days
P3 — Ongoing	Implement patch management, credential uniqueness, least privilege, and recurring configuration reviews.	IT / security owner	Ongoing

Priority labels are the proposed remediation order for this lab report, not contractual service-level commitments. Owners and due dates should be agreed by the system owner.

15 Retest Recommendations

After remediation, perform a targeted retest to verify each control is effective and that the attack chain is broken at multiple stages.

- **Attempt anonymous FTP authentication and verify it is denied; confirm wp-config.php is not readable through FTP.**
- **Verify the exposed database credential has been rotated and cannot be used by unauthorized parties.**
- **Confirm WordPress core and the Google Maps plugin are updated to supported, non-vulnerable versions; re-test the relevant SQL injection safely.**
- **Confirm the webmaster password has been changed, is unique, and no longer authenticates to SSH if SSH access is not required.**
- **Confirm sudo -l for webmaster lists only approved, minimal commands and does not permit arbitrary root command execution.**
- **Verify directory listing is disabled on wp-content/uploads and review public files for sensitive content.**
- **Review whether XML-RPC and Webmin are necessary; verify access restrictions if retained.**
- **Re-run service enumeration and compare exposed ports and versions against the original baseline.**
- **Capture retest evidence and mark each finding as Remediated, Partially Remediated, or Open.**

16 Conclusion

The HF2019 Linux lab was successfully compromised from initial unauthenticated enumeration to root-level access. The principal attack path relied on anonymous FTP exposure, a WordPress SQL injection that disclosed a user hash, a recoverable and reused password, and unrestricted sudo permissions. The combination — not any one banner or version string alone — enabled the final impact.

The highest-priority actions are to remove unrestricted sudo, rotate exposed credentials, disable anonymous FTP, protect WordPress configuration files, and patch the vulnerable WordPress component. The environment should be retested after changes.

> **This report reflects only the evidence supplied from the recorded lab run. It is not an independent certification of the host or a statement that remediation has been completed.**

17 Appendices

Appendix A — Command & Evidence Register

Activity	Command / evidence	Result
Network configuration	ip -br addr; ip route	Kali 10.0.2.12/24; gateway 10.0.2.1
Host discovery	sudo arp-scan --localnet	Target 10.0.2.6 responded
Port / service scan	nmap -sC -sV -oN hf2019-initial.txt 10.0.2.6	FTP, SSH, HTTP, Webmin detected
FTP access	ftp 10.0.2.6; anonymous; ls -la; get wp-config.php	Anonymous access and configuration retrieval confirmed
Web fingerprint	whatweb http://10.0.2.6/	Apache and WordPress identified
WordPress scan	wpscan --url http://10.0.2.6/ --enumerate p --plugins-detection passive	WordPress 5.2.3; plugin and endpoint observations
SQL injection validation	msf auxiliary/admin/http/wp_google_maps_sqli	webmaster hash retrieved
Hash recovery	john --wordlist=... hashxx.txt; john hashxx.txt --show	Password recovered
SSH access	ssh webmaster@10.0.2.6	Login succeeded
Privilege review	sudo -l; sudo su	Unrestricted sudo and root shell confirmed

Appendix B — Sensitive Evidence Handling

The assessment recovered a database secret, a WordPress password hash, a plaintext password, and two lab flags. These values are intentionally not reproduced in the main report. Store any required raw evidence in an access-controlled location and redact secrets before external publication. Rotate any credential that could be reused outside the disposable lab.

Appendix C — Severity Definitions Used

Severity	Qualitative definition
CRITICAL	A demonstrated condition enables full administrative control or similarly severe impact with limited additional barriers.
HIGH	A demonstrated weakness materially enables unauthorized access, sensitive-data exposure, or a major step toward compromise.
MEDIUM	A weakness creates meaningful exposure but requires additional conditions or has more limited demonstrated impact.
LOW	A limited-risk weakness or information disclosure with minimal direct impact.
INFORMATIONAL	A useful observation that is not, by itself, a confirmed vulnerability.

Appendix D — Limitations & Assumptions

- The target is treated as an authorized, intentionally vulnerable training lab based on the user-provided context.
- No raw Nmap output file, target logs, configuration backups, or independent host access was available to the report author.
- Plugin version was not identified by WPScan; the SQL injection finding is based on the successful module output supplied.
- Webmin, XML-RPC, and all upload subdirectories were not fully assessed.
- No remediation was performed or verified as part of the supplied evidence.
- The report contains qualitative severity labels and no calculated CVSS vector.

Appendix E — Restricted Lab Flags

> **Restricted**
> **For lab recordkeeping only. Keep this appendix restricted and remove it before sharing the report beyond authorized lab stakeholders.**

— End of Report —